

Tema 11. Universalidad.

11.1. Numeración de programas

Definición 11.1.1. Índice de un conjunto de funciones.

Un índice de un conjunto contable de funciones $\mathbf{F}$ es cualquier función sobreyectiva h :

$\mathbb{N} \rightarrow \mathbf{F}$. Un índice de una función $f \in \mathbf{F}$ bajo el índice h es cualquier $i \in \mathbb{N}$ tal que $h(i) = f$.

Para codificar programas, necesitamos introducir los conceptos de codificación y decodificación de Gödel. Estas funciones se definen utilizando el conjunto de todos los vectores de números naturales.

$$\mathbb{N}^* = \bigcup_{n=0}^{\infty} \mathbb{N}^n$$

Nota. $\mathbb{N}^0 = \{()\}$.

Definición 11.1.2. Función de emparejamiento de Cantor (Cantorización de $\mathbb{N}^2$). La función de emparejamiento de Cantor establece una biyección entre $\mathbb{N}$ y $\mathbb{N}^2$.

$$\sigma_1^2 : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$$

$$\sigma_1^2(x, y) = \frac{(x + y)(x + y + 1)}{2} + y$$

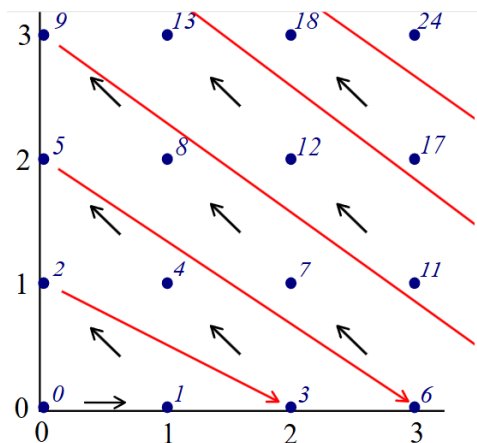
$$\sigma_{2,2}^1 : \mathbb{N} \rightarrow \mathbb{N} \times \mathbb{N}$$

$$\sigma_{2,2}^1(z) = z - \sigma_1^2(d(z), 0)$$

$$\sigma_{2,1}^1 : \mathbb{N} \rightarrow \mathbb{N} \times \mathbb{N}$$

$$\sigma_{2,1}^1(z) = d(z) - \sigma_{2,2}^1(z)$$

donde $d(z) = \min\{t \in \mathbb{N} : \sigma_1^2(t, 0) > z\} - 1$



Definición 11.1.3. Funcion de tupla de Cantor. (Cantorizacion de $\mathbb{N}^{k+1}$). Cada funcion de tupla de Cantor establece una funcion biyectiva entre $\mathbb{N}$ y $\mathbb{N}^{k+1}$, $k \in \mathbb{N}_0$.

$$\begin{aligned}\sigma_1^{k+1} &: \mathbb{N}^{k+1} \rightarrow \mathbb{N} \\ \sigma_1^1(x) &= x \\ \sigma_1^{k+1}(x_1, \dots, x_{k+1}) &= \sigma_1^2(\sigma_1^k(x_1, \dots, x_k), x_{k+1})\end{aligned}$$

$$\begin{aligned}\sigma_{k+1,p}^1 &: \mathbb{N} \rightarrow \mathbb{N} \\ \sigma_{k+1,p}^1(z) &= \begin{cases} \sigma_{k,p}^1(\sigma_{2,1}^1(z)) & \text{if } 1 \leq p \leq k \\ \sigma_{2,2}^1(z) & \text{if } p = k + 1 \end{cases}\end{aligned}$$

Nota. La tupla de Cantor es una familia de infinitas funciones biyectivas contables.

Ejemplo 11.1.1. Codificación y decodificación de Cantor

```
octave> cantorencoding(4, 5)
ans = 50

octave> cantordecoding(50,2)
ans =
4 5
```

Definición 11.1.4. Codificación de Gödel de $\mathbb{N}^*$ (godelizacion de $\mathbb{N}^*$)

La codificación y decodificación de Gödel establece una biyeccion entre $\mathbb{N}$ y $\mathbb{N}^* = \bigcup_{n=0}^{\infty} \mathbb{N}^n$.

$$\begin{aligned}\Gamma &: \mathbb{N}^* \rightarrow \mathbb{N} \\ \Gamma(\underline{x}) &= \begin{cases} 0 & \text{if } |\underline{x}| = 0 \text{ (i.e. } x = ()) \\ \sigma_1^2(|\underline{x}| - 1, \sigma_1^{|\underline{x}|}(\underline{x})) + 1 & \text{if } |\underline{x}| > 0 \end{cases}\end{aligned}$$

$$\begin{aligned}\gamma &: \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N} \\ \gamma(z, k) &= \begin{cases} 0 & \text{if } z = 0 \vee k > \lambda(z) \\ \sigma_{\lambda(z),k}^1(\sigma_{2,2}^1(z - 1)) & \text{otherwise} \end{cases}\end{aligned}$$

donde $\lambda(z)$ se refiere a la longitud del vector obtenido al decodificar z , a saber $\lambda(z) = m$ iff $\Gamma(\underline{x}) = z \wedge \underline{x} \in \mathbb{N}^m$ o, en términos de codificación previa:

$$\lambda(z) = \begin{cases} 0 & \text{if } z = 0 \\ \sigma_{2,1}^1(z - 1) + 1 & \text{if } z > 0 \end{cases}$$

Nota. De hecho, la codificación de Gödel enumera $\mathbb{N}^*$:

0	1	2	3	4	5	6	7	8	9	10	...
()	(0)	(0,0)	(1)	(0,0,0)	(1,0)	(2)	(0,0,0,0)	(1,0,0)	(0,1)	(3)	...

11.1.1. Codificación de programas While.

Definición 11.1.5. Codificación de sentencias While.

$$sent2N : \text{CODE}^1 \rightarrow \mathbb{N} \text{ with } \check{\text{CODE}}^1 = \mathcal{L}((N, T, P, < \text{sentence} >))$$

$$sent2N(c) = \begin{cases} 5(i-1) & \text{if } c = X_i := 0 \\ 5\sigma_1^2(i-1, j-1) + 1 & \text{if } c = X_i := X_j \\ 5\sigma_1^2(i-1, j-1) + 2 & \text{if } c = X_i := X_j + 1 \\ 5\sigma_1^2(i-1, j-1) + 3 & \text{if } c = X_i := X_j - 1 \\ 5\sigma_1^2(i-1, code2N(b)) + 4 & \text{if } c = \text{while } X_i \neq 0 \text{ do } b \text{ od} \end{cases}$$

Ejemplo 11.1.2. Codificación de sentencias While.

```
octave> sent2N('X2:=0')
ans = 5

octave> sent2N('X2:=X1+1')
ans = 7

octave> sent2N('while X2!=0 do X1:=0 od')
ans = 9
```

Definición 11.1.6. Codificación de código While. Dado $c = s_1; \dots; s_m \in \text{CODE}$, donde cada s_i es una sentencia, $code2N(c) = \Gamma(sent2N(s_1), \dots, sent2N(s_m)) - 1$

Definición 11.1.7. Codificación de programas While.

Dada $Q = (n, c) \in \text{WHILE}$,

$$while2N : \text{WHILE} \rightarrow \mathbb{N}$$

$$while2N(Q) = \sigma_1^2(n, code2N(c))$$

Ejemplo 11.1.3. Codificación de un código y programa While.

```
octave> CODE2N('while X2!=0 do X1:=0 od')
ans = 54

octave> WHILE2N(1, 'while X2!=0 do X1:=0 od')
ans = 1594
```

Ejemplo 11.1.4. Codificación del programa 'double'

```
double = (1, s)
s :
  X2 := X1;
  while X2 ≠ 0 do
    X1 := X1 + 1;
    X2 := X2 - 1
  od
```

$$code2N(s) \approx 1 \times 10^{39}$$

$$while2N(double) \approx 5 \times 10^{77}$$

11.1.2. Decodificación de programas While.

Definición 11.1.8. Tipo de funcion sentencia.

$$senttype : \mathbb{N} \rightarrow \{0, 1, 2, 3, 4\}$$

$$senttype(z) = z \bmod 5$$

Definición 11.1.9. Funciones LHS y RHS

$$LHS, RHS : \mathbb{N} \rightarrow \mathbb{N}$$

$$LHS(z) = \begin{cases} 1 + \frac{1}{5}z & \text{if } senttype(z) = 0 \\ 1 + \sigma_{2,1}^1 \left(\frac{1}{5}(z - senttype(z)) \right) & \text{if } senttype(z) \in \{1, 2, 3, 4\} \end{cases}$$

$$RHS(z) = 1 + \sigma_{2,2}^1 \left(\frac{1}{5}(z - senttype(z)) \right)$$

Definición 11.1.10. Decodificar una sentencia While.

$$N2sent : \mathbb{N} \rightarrow \text{CODE}$$

$$N2sent(z) = \begin{cases} X_i := 0 & \text{if } senttype(z) = 0 \\ X_i := X_j & \text{if } senttype(z) = 1 \\ X_i := X_j + 1 & \text{if } senttype(z) = 2 \\ X_i := X_j - 1 & \text{if } senttype(z) = 3 \\ \text{while } X_i \neq 0 \text{ do } N2code(j-1) \text{ od} & \text{if } senttype(z) = 4 \end{cases}$$

where $i = LHS(z)$, $j = RHS(z)$

Ejemplo 11.1.5. Decodificación de sentencias While

```

octave> N2sent(5)
ans = X2:=0

octave> N2sent(7)
ans = X2:=X1+1

octave> N2sent(9)
ans = while X2!=0 do X1:=0 od

```

Definición 11.1.11. Decodificación de códigos While.

$N2code : \mathbb{N} \rightarrow \text{CODE}$

$$N2code(z) = \begin{cases} N2sent(\gamma(z+1, 1)) & \text{if } z = 0 \\ N2sent(\gamma(z+1, 1)); \dots; N2sent(\gamma(z+1, \lambda(z+1))) & \text{if } z > 0 \end{cases}$$

Definición 11.1.12. Decodificar programas While.

$N2while : \mathbb{N} \rightarrow \text{WHILE}$

$$N2while(z) = (\sigma_{2,1}^1(z), N2code(\sigma_{2,2}^1(z)))$$

Ejemplo 11.1.6. Decodificación de códigos y programas While.

```

octave> N2CODE(54)
ans = while X2!=0 do X1:=0 od

octave> N2WHILE(1594)
ans = (1, while X2!=0 do X1:=0 od)

```

Definición 11.1.13. REC^n

$$\text{REC}^n = \{f \in \text{REC} : f : \mathbb{N}^n \rightarrow \mathbb{N}\}.$$

Nota. REC^n es el conjunto de funciones REC con aridad n.

Definición 11.1.14. $F(\text{WHILE})^n$

$$F(\text{WHILE})^n = \{f \in F(\text{WHILE}) : f = F_{(n,s)} \wedge (n, s) \in \text{WHILE}\}.$$

Definición 11.1.15. Indexación de REC.

$$h : \mathbb{N} \rightarrow F(\text{WHILE})$$

$$h(z) = F_{N2while(z)}$$

Dada el teorema de equivalencia, la funcion es tambien una indexación de REC

Definición 11.1.16. Indexación de REC^n

$$h : \mathbb{N} \rightarrow F(\text{WHILE})^n$$

$$h(z) = F_{(n, N2code(z))}$$

Dado el teorema de equivalencia, la funcion es tambien una indexación de REC^n

11.2. Funcion universal

Definición 11.2.1. Funcion universal

Dado un conjunto contable de funciones $\mathbf{F}$ que verifica $f \in \mathbf{F} \Rightarrow f : \mathbb{N}^n \rightarrow \mathbb{N}, n \geq 0$,

$U[\mathbf{F}] : \mathbb{N}^{n+1} \rightarrow \mathbb{N}$ es universal para $\mathbf{F}$ si y solo si

$\exists h : \mathbb{N} \rightarrow \mathbf{F}$ que es una indexación de
 $\mathbf{F} : U[\mathbf{F}](i, \underline{x}) = h(i)(\underline{x}) \forall i \in \mathbb{N} \wedge \forall \underline{x} \in \mathbb{N}^n$.

Nota. $U[\mathbf{F}]$ es la funcion universal de $\mathbf{F}$ bajo la indexación de h.

Teorema 11.2.1. $U[REC^n] \in REC^{n+1}$

Asumiendo

$$\exists U \in \text{WHILE} : F_U = U[REC^n] \Rightarrow$$

$$U[REC^n] \in F(\text{WHILE})^{n+1} \Rightarrow$$

$$U[REC^n] \in REC^{n+1}$$

tiene que estar demostrado que $\exists U \in \text{WHILE} : F_U = U[REC^n]$

Nota. U será denotado como el programa universal.

11.3. Programa Universal

Definición 11.3.1. Programa universal.

$$U = (k+1, s)$$

$s :$

(* X1 stores the code of the program *)
 (* X2,...,Xk+1 store the arguments of the program *)
 (* arguments are encoded in a single Gödel number *)
 $X_{k+2} := \text{godelk}(X_2, \dots, X_{k+1});$
 (* variables after simulating the program *)
 (* code in first argument, variables in second argument *)
 $X_{k+2} := \text{simulate}(X_1, X_{k+2});$
 (* first variable is the output *)
 $X_1 := \text{degodel}(X_{k+2}, 1)$

```

simulate = (2, s)
s :
    X1 := X1 + 1;
    while X1 ≠ 0 do
        (* extract first sentence and check type *)
        X3 := degodel(X1,1);
        (* check if assignment or loop *)
        if senttype(X3) ≤ 3 then
            (* change variable and remove *)
            X2 := execute(X3,X2);
            X1 := pop(X1)
        else
            (* value of control variable *)
            X4 := lhs(X3);
            if degodel(X2,X4) ≠ 0 then
                (* push body of the loop to top of the stack *)
                X4 := rhs(X3);
                X1 := merge(X1, X4)
            else
                (* remove sentence from the stack *)
                X1 := pop(X1)
            fi
        fi
    od
    (* return the encoded vector of variables *)
    X1 := X2

```

```

execute = (2, s)
s :
  (* X1 = sentences, X2 = variables *)
  X3 := senttype(X1);
  (* index of target variable *)
  X4 := lhs(X1);
  if X3 = 0 then
    X1 := replace(X2,X4,0)
  else
    (* value of source variable *)
    X5 := rhs(X1);
    X5 := degodel(X2,X5);
    if X3 = 1 then
      X1 := replace(X2,X4,X5)
    fi
    if X3 = 2 then
      X5 := X5 + 1;
      X1 := replace(X2,X4,X5)
    fi
    if X3 = 3 then
      X5 := X5 - 1;
      X1 := replace(X2,X4,X5)
    fi
  fi
pop = (2, s)
s :
  (* X1 = sentences, X4 = number of sentences *)
  X4 := length(X1);
  (* return 0 if the program is just one sentence long *)
  if X4 = 1 then
    X2 := 0
  else
    (* godel of sentences except the first one *)
    X2 := godelk(degodel(X1,2));
    X4 := X4 - 1;
    X4 := X4 - 1;
    while X4 ≠ 0 do
      X2 := godelk(X2,degodel(X1,X3));
      X3 := X3 + 1;
      X4 := X4 - 1
    od
    X1 := X2
  fi

```


merge = (2, *s*)

s :

(* X1 = sentences, X2 = sentences to move to the top of the stack *)

if length(X2) > 0 **then**

 X4 := godelk(degodel(X2,1));

 X3 := 2;

 X5 := length(X2);

 X5 := X5 - 1;

while X5 ≠ 0 **do**

 X4 := godelk(X4,degodel(X2,X3));

 X3 := X3 + 1;

 X5 := X5 - 1

od

(* once all the sentences of X2 are on top, we introduce the rest of the program *)

X5 := length(X1);

X5 := X5 - 1;

while X5 ≠ 0 **do**

 X4 := godelk(X4,degodel(X1,X3));

 X3 := X3 + 1;

 X5 := X5 - 1

od

X1 := X4

replace = (3, *s*)

s :

(* X1 = variables, X2 = variable index, X3 = value *)

X7 := 2;

if X2 = 1 **then**

 X6 := godelk(X3)

else

 X6 := godelk(degodel(X1,1))

fi

X4 := length(X1);

(* check how many variables are represented *)

if X4 >= X2 **then**

 X5 := X4

else

 X5 := X2

fi

X4 := X4 - 1;

while X5 ≠ 0 **do**

 (* fill with zeros when needed *)

if X4 = 0 **then**

if X7 = X2 **then**

 X6 := godelk(X6,X3)

else

 X6 := godelk(X6,0)

fi

else

if X7 = X2 **then**

 X6 := godelk(X6,X3)
